

목차

1. 초록
 2. 서론
 3. 배경 조사
 - 가. 관련 접근방법 및 기술의 장단점 분석
 - 나. 프로젝트 개발환경
 4. 목표·문제 정의 및 요구사항
 5. 접근방법
 6. 프로젝트 아키텍처
 - 가. 아키텍처 구성도
 - 나. 아키텍처 설명
 7. 구현 명세
 - 가. 입출력 인터페이스
 - 나. 모듈 간 통신 인터페이스
 - 다. 모듈 구성
 8. 현재 진행 상황
 9. 향후 작업
 10. 업무 분담 및 담당
 11. 일정
- 부록. 상세 구현 명세

1. 초록

Foodify는 모바일 기기로 촬영한 한 끼 음식 사진을 분석하여 음식 종류와 제공량을 추정하고, 이에 따른 열량과 영양성분 및 개인화된 식단 피드백을 제공하는 애플리케이션이다. OpenAI API의 gpt-4o-mini를 사용할 모델로 선정했으며, 영양성분은 공공 데이터베이스의 기준값으로 서버에서 계산한다. Flutter Android 앱과 Java 21 Spring Boot 서버를 사용하고, 갤럭시 S23을 기준 시험 기기로 선정했다.

회원별 촬영 기록과 분석 결과를 자동 저장하고, 완료된 기록을 영양 합계에 반영한다. 사용자가 피드백을 요청하면 직접 입력한 목표와 최근 1일·7일 기록을 바탕으로 식생활 참고 안내를 제공한다. 음식 종류 수는 서비스 제한이 아닌 평가 자료의 범위로 관리한다. 자동 음식량 추정은 단순한 형상의 음식 3종으로 실측 검증을 시작하고, 검증된 조건부터 적용한다.

2. 서론

식단을 관리하려는 사용자는 음식명과 섭취량을 직접 입력하고, 해당 음식의 영양성분을 찾아 기록해야 한다. 이러한 입력 과정을 간소화하기 위해 음식 사진을 활용할 수 있지만, 음식 종류를 인식하는 것만으로 실제 영양 섭취량을 계산하기는 어렵다. 같은 음식이라도 제공량과 조리법에 따라 열량과 영양성분이 달라지기 때문이다.

본 프로젝트는 음식 인식, 음식량 추정, 영양성분 계산, 식단 기록 및 개인화 피드백을 연결하는 모바일 서비스를 제안한다. 별도의 음식 인식 모델을 처음부터 학습하기보다 외부 언어 모델 서비스를 활용하고, 영양성분 계산에는 공공 데이터베이스의 기준값을 사용한다.

주요 대상은 식사 기록과 영양 관리가 필요한 모두이다. 초기 개발에서는 시장 조사보다 핵심 기능의 구현 가능성과 분석 결과의 정확도 확인에 집중한다.

3. 배경 조사

가. 관련 접근방법 및 기술의 장단점 분석

접근방법	장점	한계 및 적용 방향
사용자 직접 입력	음식명과 중량을 알고 있다면 명확한 기록 가능	입력 부담이 있으므로 분석 결과 수정 기능에 활용

접근방법	장점	한계 및 적용 방향
음식 인식 모델 직접 학습	선정한 음식군에 맞춰 모 델 개선 가능	데이터 정제·학습·운영 부담을 고려하여 초 기에는 외부 언어 모델 API 활용에 집중
언어 모델 기반 이미지 분석	음식명·재료 후보를 함께 추정하고 설명 생성 가능	오인식 및 수치 추정 오류가 있어 결과 검 증과 사용자 수정 필요
증강현실 및 공 간 정보 활용	실제 크기를 추정하는 보 조 정보 확보 가능	ARCore·Depth API 지원 여부를 각각 확인하 고 부피 계산 방식을 별도로 검증
공공 영양성분 데이터베이스 활용	수치의 자료원과 계산 근 거를 추적하기 쉬움	음식명·조리법·제공량을 적절한 데이터 항목 과 연결해야 함
검색 증강 생성	검색된 음식·재료 정보를 근거로 분석과 설명 보조 가능	음식 인식·중량 추정의 정확도 검증과 함께 활용

한국 음식 분석 자료로는 AI Hub의 음식 이미지·영양정보 데이터를 활용할 수 있다. 본 프로젝트에서는 주로 언어 모델의 음식 인식 결과를 확인하는 검증 자료로 검토한다. 해 외 자료 중 Food-101은 음식 분류 검증에, Nutrition5k는 중량과 영양성분 추정 검증에 활용할 수 있다.

공식 영양 자료로 음식·원재료성 식품·가공식품·통합 식품영양성분·건강기능식품 CSV 5개 를 확보했다. 음식 자료를 우선 검색 대상으로 삼고, 재료와 제품 자료는 입력 근거에 따 라 연결한다. 원본 파일과 행, 기준량, 단위, 출처 및 기준일을 보존하여 영양값의 근거를 추적한다. 빈 값은 미상으로 관리하며 g와 ml를 구분한다.

Google 공식 지원 목록에서 갤럭시 S23의 ARCore 및 Depth API 지원을 확인했다. 이를 활용해 카메라 위치, 평면, 깊이 및 신뢰도 정보를 수집하는 실험을 진행한다. 지원 사실 과 음식 부피 측정 정확도는 구분하며, 실제 기기의 Android 버전과 깊이 수집 결과를 추가 확인한다. 부피·중량 계산에는 음식 영역, 기준면, 높이 및 밀도 등의 근거를 결합한 다. 출처: <https://developers.google.com/ar/devices?hl=ko> 및 <https://developers.google.com/ar/develop/java/depth/raw-depth>

나. 프로젝트 개발환경

항목	개발환경 및 계획
모바일 애플리케이션	Flutter, Riverpod, go_router, Dio
서버	Java 21, Spring Boot, Flyway
우선 지원 운영체제	Android 우선
실기기	갤럭시 S23 선정, 실제 Android 버전 확인 예정
현재 개발 컴퓨터	윈도 환경
Android 빌드 환경	Windows, Android Studio 및 Android SDK
음식 이미지 분석	OpenAI API, gpt-4o-mini 선정
영양성분 데이터	공식 CSV 5개 확보, 출처·단위·버전 관리형 DB
데이터 저장	로컬 H2 검증, 배포 AWS RDS MySQL
배포 환경	EC2, HTTPS 프록시, 비공개 S3 계획

Windows에서 Flutter와 Android SDK를 사용한다. 기준 시험 기기는 갤럭시 S23이며, USB 디버깅 연결과 APK 실행·카메라·AR 측정을 검증한다. 모바일은 Riverpod, go_router, Dio 및 운영체제 보안 저장소를 사용한다. 서버는 Java 21, Spring Boot, Flyway 기반이다. 로컬 검증에는 H2 MySQL 호환 모드를 사용하고 배포 DB는 AWS RDS MySQL로 계획한다.

4. 목표·문제 정의 및 요구사항

본 프로젝트의 핵심 문제는 **음식 사진에서 음식 종류뿐 아니라 양을 추정하고, 그 결과를 근거 있는 영양성분 계산과 지속적인 식단 기록으로 연결하는 것이다.**

식전 촬영만 고려하므로 시스템이 직접 추정하는 대상은 '제공된 음식량'이다. 이를 모두 섭취했다는 가정하에 식단 기록에 반영하며, 실제 섭취량과 차이가 있으면 사용자가 수정할 수 있도록 설계한다.

구분	요구사항
촬영	식전 한 끼 전체, 짧은 안내 촬영, 대표 이미지 최대 3장
음식 인식	LLM API로 음식·재료 분석, 대표 음식군으로 성능 평가

구분	요구사항
음식량 추정	음식 3종 실측 실험 후 검증된 범위 적용, 사용자 보정 제공
영양 계산	추정량과 기준 영양정보를 이용해 열량 및 3대 영양소 계산
자동 저장	회원별 촬영 기록과 분석 결과 자동 저장
실패 처리	분석 실패 기록을 '분석 미완료'로 유지하고 영양 합계에서 제외
기록 수정	음식명과 양을 수정하면 영양성분 재계산
이력 조회	최근 1일·1주 식단과 영양 합계 확인
피드백	사용자 요청 시 목표와 식단 이력을 반영한 안내 생성
회원 관리	이메일·비밀번호 가입, 토큰 인증 및 회원별 자료 접근 관리

분석 결과에는 추정치임을 표시하고, 계산에 사용한 음식 데이터와 자료원을 저장한다. 피드백은 사용자 목표와 기록된 식단에 근거한 식생활 참고 안내로 제공한다.

5. 접근방법

개발은 음식량 추정의 실현 가능성을 먼저 확인하고, 그 결과를 서비스 흐름에 연결하는 순서로 진행한다.

1. 기기 및 개발환경 확인

갤럭시 S23을 기준으로 앱 설치·카메라 촬영을 확인한다. 공식 지원 조사 결과에 더해 ARCore 실행 상태와 실제 깊이 데이터 수집 가능 여부를 기기에서 검증한다.

2. 음식 인식 기능 구현

대표 이미지 최대 3장을 서버에서 OpenAI API로 전달하고 음식명·재료 후보·참고 중량·불확실 항목을 구조화해 받는다. gpt-4o-mini를 사용하도록 운영 환경변수를 지정하며, 응답 형식과 단위 검증은 서버에서 수행한다.

3. 음식량 추정 실험

음식 3종별 10가지 제공량을 각각 2회 촬영한다. 동일 제공량의 반복 촬영이 보정용과 검증용 양쪽에 섞이지 않도록 분리한다. 검증 중량 상대오차 중앙값 25% 이하, 자동 결과 확보율 80% 이상을 초기 채택 목표로 삼는다. 이는 실험 기준이며 전체 음식의 보장 정확도와 구분한다.

4. 영양성분 연결 및 계산

CSV를 정규화하고 중복·충돌·누락값을 점검한 뒤 버전별 영양 DB에 적재한다. 음식 후보를 검색하고 필요하면 검색 결과 안에서 LLM이 선택을 보조한다. 현재 구현은 g 기준 계산부터 제공하며 ml와 제공량 계산은 후속 확장한다.

5. 기록 및 피드백 제공

분석 결과를 자동 저장하고, 사용자가 요청하면 식단 이력에 기반한 피드백을 생성한다.

영양값은 기준 영양량에 기록량과 기준량의 비율을 곱해 계산한다. 기록량과 기준량은 같은 단위를 사용한다. 음식 부피를 중량으로 환산할 때는 밀도 또는 검증된 환산 근거를 적용한다. 필수 영양값이나 양의 근거가 부족하면 확인 필요 상태를 유지하고 사용자 보정으로 이어간다.

6. 프로젝트 아키텍처

가. 아키텍처 구성도

다음 구성은 구현된 기본 흐름과 배포·AR 확장 계획을 함께 나타낸다.

[회원]

- | 촬영·기록 조회·피드백 요청



[Foodify Android 애플리케이션]

- |— 촬영 안내 및 이미지 수집
- |— Kotlin ARCore 지원 확인 및 공간 정보 수집 확장
- |— 분석 상태·결과 표시
- |— 식단 조회·수정
- |
- | 암호화된 웹 통신



[스프링 부트 서버]

- |— 회원 식별 및 접근 관리
- |— 촬영 접수 및 분석 작업 관리
- |— 음식 인식 연동
- |— 참고 중량 처리 및 사용자 보정
- |— 영양정보 검색 및 계산
- |— 식단 기록 및 영양 집계
- |— 요청 기반 개인화 피드백
- |
- |— [OpenAI API gpt-4o-mini 선정]
- |— [버전 관리형 공공 영양 DB]
- |— [로컬 H2 검증 / 배포 RDS MySQL 계획]
- |— [로컬 사진 저장 / 비공개 S3 배포 계획]

음식량 추정 연산을 모바일과 서버 중 어디에서 수행할지는 기기 실험 후 확정한다. 촬영 및 기기 센서 접근은 모바일에서 수행한다.

나. 아키텍처 설명

모바일 애플리케이션은 촬영 안내, 입력 수집, 분석 상태 표시 및 기록 조회를 담당한다. 기기에서 수집한 이미지와 필요한 측정 정보를 서버에 전달한다.

서버는 분석 작업을 관리하고 음식 인식, 음식량 추정, 영양 계산 결과를 결합한다. 초기에는 하나의 서버 안에서 기능별 모듈을 구분하는 구조를 사용한다. 외부 언어 모델과 영양 데이터에 대한 접근도 서버에서 관리한다.

촬영 접수 후 식단 기록과 분석 작업을 저장하고 서버 작업 처리기가 분석을 실행한다. 앱은 분석 화면에서 3초 간격으로 상태를 조회한다. 동일 업로드는 요청 식별자로 중복

을 막고, 기록 버전을 비교하여 늦게 도착한 분석 결과가 사용자 수정값을 덮어쓰는 상황을 방지한다.

배포는 EC2의 Spring Boot 서버, RDS MySQL, 비공개 S3를 기본안으로 한다. 현재 로컬 DB와 사진 저장을 지원하고 S3 연결 코드를 준비했다. 실제 AWS 자원 생성과 배포 검증은 후속 단계이다. API 키는 서버 환경변수로 주입하며 배포 시 Parameter Store SecureString과 EC2 IAM 역할을 연결하는 방안을 적용할 예정이다.

7. 구현 명세

가. 입출력 인터페이스

기능	입력	출력
촬영 접수	회원 인증 정보, 이미지, 촬영 시각, 수집 가능한 측정 정보	기록 식별자, 분석 작업 식별자
음식 인식	음식 이미지	음식 후보, 재료 후보, 인식 불확실 항목
음식량 추정	이미지, 측정 정보, 추정에 필요한 보조 정보	추정량, 단위, 추정 방법 또는 실패 사유
영양 계산	음식 데이터 식별자, 추정 중량	열량·탄수화물·단백질·지방, 자료원
기록 조회	조회 기간	식단 목록, 분석 상태, 영양 합계
기록 수정	수정할 음식 또는 양	재계산된 영양 결과
피드백 요청	조회 기간, 사용자 목표	식단 기록에 근거한 피드백

음식 후보나 양이 불명확하면 미확정 사유를 포함한 분석 미완료 상태를 반환하고, 추가 확인이나 수정에 필요한 정보를 제공한다.

나. 모듈 간 통신 인터페이스

모바일과 서버는 /api/v1 경로의 REST API로 통신한다. 일반 데이터는 JSON, 촬영 접수는 multipart 형식으로 전달한다. 배포는 HTTPS를 사용하고 로컬 개발 연결은 개발 설정

으로 구분한다. 서버의 OpenAPI 명세를 공유하여 요청과 응답 형식을 관리한다.

서버 내부 모듈은 초기에는 직접 메서드 호출로 연결한다. 분석 단계별로 입력과 출력 형식을 구분하여 음식 인식 서비스나 양 추정 방식을 변경해도 다른 기능의 수정 범위를 줄인다.

외부 언어 모델의 응답은 서버에서 필수 항목, 단위 및 값의 형식을 검증한 뒤 다음 단계로 전달한다. 영양 계산은 검색된 기준 데이터와 검증된 양을 바탕으로 서버에서 수행한다.

분석은 DB에 상태를 저장하는 서버 내부 작업 처리기로 실행한다. 일시 오류는 최대 3회 시도하며 재시도 간격은 5초~20초이다. 서버 재시작 후 남은 작업을 복구하고, 월 요청 수 한도에 도달하면 대기 처리한다. 요청 수 제한은 원화 비용 상한과 구분하여 운영 시 실제 사용량을 확인한다.

다. 모듈 구성

모듈	주요 책임
회원 관리	회원 식별, 목표 저장, 기록 접근 권한 확인
촬영 관리	촬영 안내, 이미지 및 기기에서 지원하는 공간 정보 수집
분석 작업 관리	접수·진행·완료·미완료 상태 관리
음식 인식	언어 모델 호출 및 음식·재료 후보 정리
음식량 추정	선택한 방식에 따른 양 추정 및 실패 판정
영양정보 검색	음식 후보와 기준 데이터 연결
영양 계산	단위 변환, 추정량에 따른 영양성분 계산
식단 기록	자동 저장, 조회, 수정, 삭제 및 기간별 집계
피드백	요청 시 사용자 목표와 기록을 반영한 안내 생성

8. 현재 진행 상황

2026년 9월 28일 기준, 회원 가입·로그인·토큰 갱신, 목표 설정, 사진 업로드, 분석 상태 관리, 음식 수정·영양 계산, 일간·주간 집계 및 요청 기반 피드백의 1차 코드를 구현했다. 공식 영양 CSV 변환기와 버전 관리형 DB 적재기를 추가하고 로컬 H2 환경에서 자료 적

재를 검증했다. 서버 자동 테스트 16개와 변환 기본 테스트 3개가 통과했다. 실제 OpenAI 호출, S23 실기기, MySQL-RDS-S3 배포 및 자동 부피 추정은 별도 검증 항목이다. 확정된 사항은 다음과 같다.

- 플러터와 자바 스프링 부트를 사용한다.
- Android를 우선 대상으로 하며 기준 시험 기기는 갤럭시 S23이다. Android-One UI 버전과 기기 실행 결과는 실기기 점검 시 기록한다.
- 짧은 촬영 과정을 허용하고 식전 사진만 고려한다.
- 한 끼 전체의 여러 음식을 분석 대상으로 한다. 음식 종류 수는 평가 범위로 관리하며 자동 양 추정은 음식 3종의 초기 실측 실험으로 검증한다.
- 기록은 자동 저장하고, 분석 실패 시 미완료로 유지한다.
- 피드백은 사용자가 요청할 때 생성한다.
- 이메일·비밀번호 가입과 로그인, 만료되는 접근 토큰·갱신 토큰, 회원별 접근 권한을 구현했다. 공개 운영 전 이메일 인증과 비밀번호 재설정을 확장한다.
- 개발 인원은 1명이며 첫 시연까지 약 두 달이다.

영양 DB는 원본 파일 5개, 식품 85,337개, 영양값 버전 110,939개, 원본 행 128,755개로 구성한다. 현재 검색 대상은 충돌 없는 g 기준 일반 식품 66,833개이며 그중 필수 영양값 4종이 모두 있는 항목은 55,627개이다. 동일 최신 날짜의 값 충돌 식품 2개는 검색에서 보류한다. 이 수치는 다운로드한 파일 집합 기준이며 공식 전체 DB 규모를 의미하지 않는다.

9. 향후 작업

다음 작업은 Android APK 빌드 오류 해결, S23 설치·촬영 검증, gpt-4o-mini 실제 이미지 분석 연결, 로컬 MySQL 검증 및 음식량 실측 실험이다. 기본 서비스 흐름과 데이터 계산을 우선 안정화한 뒤 검증된 AR 방식을 통합한다.

한 끼 사진 50세트 중 30세트는 개발용, 20세트는 최종 평가용으로 분리한다. 음식 인식 정확도 85% 이상과 검출 재현율 90% 이상을 초기 목표로 둔다. 중량은 저울 실측값과 비교하고 자동 추정과 사용자 보정 결과를 각각 보고한다. 영양 계산은 DB 기준값과 식의 일치를 검증하며 실제 음식 성분의 정확도와 구분한다.

첫 시연 이후에는 이메일 인증·비밀번호 재설정, 지원 기기 확대, 음식별 자동 양 추정 개선, ml·제공량 계산 확장, 식후 잔반 반영 및 스토어 공개 준비를 진행한다.

10. 업무 분담 및 담당

본 프로젝트는 1인 개발로 진행하며 전체 업무를 개발자가 담당한다.

업무	세부 내용	담당
기획 및 설계	요구사항, 아키텍처, 데이터 구조 정의	본인
모바일 개발	촬영, 기기 기능 연동, 결과·기록 화면	본인
서버 개발	회원 관리, 분석 처리, 기록 관리	본인
분석 기능	언어 모델 연동, 음식량 추정, 영양 계산	본인
검증	실측 자료 수집, 기능 및 오차 확인	본인
문서 및 발표	결과 정리, 계획서·보고서·시연 준비	본인

외부 언어 모델과 공개 데이터베이스를 활용하여 개발 범위를 조정하며, 음식량 추정처럼 불확실성이 높은 기능에 초기 시간을 우선 배정한다.

11. 일정

1인 개발로 주 20~30시간, 8주를 기준으로 계획한다. 아래 일정은 기능별 목표이며 현재 구현 진척을 완료 판정과 별도로 관리한다. 월 AWS·AI 예산은 5만원 이내를 목표로 실제 견적과 사용량을 확인한다.

기간	주요 작업	산출물
1주차	Android 실행, S23 연결, OpenAI 호출 및 데이터 확인	실기기 촬영·API 연결 결과
2주차	음식 3종 양 추정 실험, DB·API 명세 확정	채택 범위와 보정 흐름
3주차	회원·목표, 사진 업로드·자동 기록·상태 관리	회원별 저장 및 상태 복구
4주차	한 끼 인식, 영양 DB 검색·중량 보정·계산	촬영부터 영양 계산까지 연결

기간	주요 작업	산출물
5주차	검증된 AR 통합, 재분석·복구·집계	오류·중복·집계 검증
6주차	피드백·삭제, 클라우드 배포, 서명 APK	S23에서 배포 서버 시연
7주차	고정 평가 자료, 보안·비용·복구 시험	정확도·지연·비용 결과
8주차	회귀 테스트, 시연 리허설, 문서·발표	최종 APK와 운영 안내

2주 차 종료 시점을 음식량 추정 방식의 판단 시점으로 둔다. 측정 결과가 충분하지 않으면 자동 추정 범위나 보정 방식의 변경안을 정하고, 이후 서비스 구현 일정에 반영한다.

부록. 상세 구현 명세

가. 주요 저장 데이터

데이터	주요 항목
회원	회원 식별자, 식단 관리 목표
식단 기록	기록 식별자, 회원 식별자, 식사 시각, 이미지 위치
분석 작업	작업 식별자, 상태, 진행 단계, 실패 사유, 생성·완료 시각
음식 분석	음식 후보, 선택된 음식, 재료 후보, 분석 모델 정보
양 추정	추정량, 단위, 추정 방법, 사용자 수정 여부
영양 결과	기준 음식 식별자, 기준량, 계산 중량, 열량 및 3대 영양소, 자료원
피드백	대상 기간, 사용한 기록 기준 시점, 생성 시각, 내용

나. 분석 상태와 처리 규칙

분석 상태는 '접수', '분석 중', '완료', '분석 미완료'로 구분한다.

- 접수: 이미지와 기록이 서버에 저장된 상태이다.

- 분석 중: 음식 인식, 양 추정 또는 영양 계산이 진행 중인 상태이다.
- 완료: 영양 계산에 필요한 결과가 확보되어 기록에 반영된 상태이다.
- 분석 미완료: 필수 결과를 확보하지 못한 상태이며 영양 합계에서 제외한다.

재시도할 때는 같은 식단 기록에 분석 작업을 연결하여 중복 기록을 방지한다. 음식명이나 양을 수정하면 기존 값을 새 값으로 갱신하고 영양 합계를 다시 계산한다.

다. 피드백 생성 규칙

사용자가 피드백을 요청하면 해당 기간의 완료된 기록과 사용자 목표를 조회한다. 열량과 영양소 합계는 서버에서 계산하고, 언어 모델은 그 계산 결과를 설명하는 역할을 맡는다.

사용자가 입력한 일일 열량·탄수화물·단백질·지방 목표와 최근 1일·7일의 완료된 기록을 비교한다. 목표가 비어 있으면 기록된 구성 중심으로 설명한다. 미완료 기록과 기록 누락 가능성을 함께 안내하고, 기록 수정 후에는 피드백 기준 시점을 표시하여 재생성을 안내한다.

라. 추가 확정이 필요한 항목

- S23의 Android-One UI 버전과 실기기 APK·카메라·깊이 수집 결과
- 음식별 깊이 품질, 기준면 및 밀도 환산 근거와 자동 양 추정의 검증 범위
- g 기준 보정 흐름의 실기기 사용성 및 ml·제공량 입력 확장
- 한 끼 전체를 짧게 촬영하는 안내 흐름과 촬영 품질 기준
- 실제 MySQL-RDS 마이그레이션 검증 및 공개 운영용 이메일 인증·비밀번호 재설정
- gpt-4o-mini 실제 호출의 인식 품질·지연·비용과 영양 DB 충돌 항목 검토
- 배포 예산, 서버 운영 시간 및 사용량 한도 설정
- 사진은 기록 삭제 시까지 보관하고 계정 삭제에도 연동한다. S3 환경에서 삭제와 접근 차단을 추가 검증한다.

마 영양 데이터베이스 구성

식품 목록, 버전별 영양값, 원본 출처를 분리한다. 예를 들어 사용자가 김치찌개를 선택하면 foods에서 항목을 찾고 현재 영양값 버전을 연결하여 100g 기준값과 입력 중량으

로 계산한다. 식단에는 계산 당시 버전과 수치의 복사본을 저장하므로 기존 DB가 갱신되어도 과거 기록의 근거를 유지한다.

테이블	역할
nutrition_datasets	CSV 파일명, 인코딩, 파일 해시, 원본 행 수와 적재 시각
nutrition_source_rows	파일별 행 번호와 전체 원본 값, 연결된 영양값 버전
foods	식품 코드·이름·분류·제조사, 현재 영양값과 검색 가능 상태
food_nutrition_versions	기준량·단위, 열량·3대 영양소, 자료원·기준 일별 이력
food_aliases	음식명 검색용 표기와 별칭
food_portions	근거가 명확한 1회 참고량과 단위

현재 영양 검색·계산은 g 기준으로 제공한다. ml 기준과 건강기능식품은 출처·영양값을 보존하고 후속 기능 확장에 활용한다. 누락된 필수 영양값은 NULL로 유지하며, 같은 최신 기준일에 값이 충돌하는 항목은 검토 대상으로 분리한다. 자동 병합은 동일 식품 코드와 분류를 기준으로 수행하고 최신 데이터기준일자를 우선한다.

CSV 변환 결과는 JSONL로 생성하고 트랜잭션 적재기를 사용한다. 같은 자료를 다시 실행해도 식별자로 중복을 방지한다. 실제 배포 전 MySQL 검증을 별도로 수행하며 RDS에 동일 Flyway 마이그레이션을 적용한다.

바 참고 자료

식품의약품안전처 K-FIND DB 다운로드:

<https://various.foodsafetykorea.go.kr/nutrient/general/down/list.do>

Google ARCore 지원 기기: <https://developers.google.com/ar/devices?hl=ko>

Google ARCore Raw Depth: <https://developers.google.com/ar/develop/java/depth/raw-depth>

OpenAI 운영 가이드: <https://developers.openai.com/api/docs/guides/production-best-practices>

AWS Parameter Store: <https://docs.aws.amazon.com/systems->

[manager/latest/userguide/what-is-a-parameter.html](#)